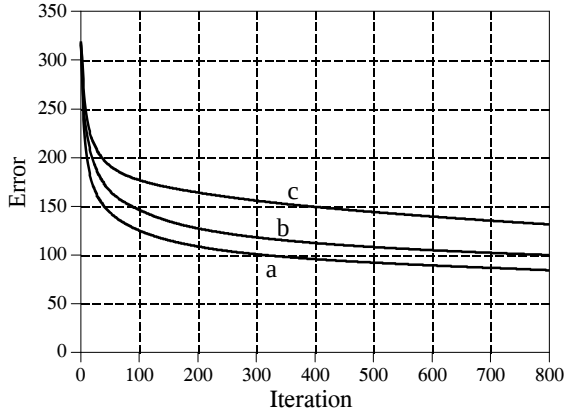

FIGURE 26-9
Neural network convergence. This graph shows how the neural network error (the value of ESUM) decreases as the iterations proceed. Three separate trials are shown, each starting with different initial weights.

record that elevation change. Using these two values, he can determine which direction is downhill. Suppose the paratrooper drops 10 cm when he moves one step in the northern direction, and drops 20 cm when he moves one step in the eastern direction. To travel directly downhill, he needs to move along each axis an amount proportional to the slope along that axis. In this case, he might move north by 10 steps and east by 20 steps. This moves him down the steepest part of the slope a distance of $\sqrt{10^2 + 20^2} = 22.4$ steps. Alternatively, he could move in a straight line to the new location, 22.4 steps along the diagonal. The key point is: *the steepest descent is achieved by moving along each axis a distance proportional to the slope along that axis.*

Subroutine 3000 implements this same steepest decent algorithm for the network weights. Before entering subroutine 3000, one of the example images has been applied to the input layer, and the information propagated to the output. This means that the values for: X1[ ], X2[ ] and X3 are all specified, as well as the current weight values: WH[,] and WO[ ]. In addition, we know the error the network produces for this particular image, ELET. The hidden layer weights are updated in lines 3050 to 3120, while the output layer weights are modified in lines 3150 to 3190. *This is done by calculating the slope for each weight, and then changing each weight by an amount proportional to that slope*. In the paratrooper case, the slope along an axis is found by moving a small distance along the axis (say, Δx), measuring the change in elevation (say, ΔE), and then dividing the two ($\Delta E/\Delta x$). The slope of a neural network weight can be found in this same way: add a small increment to the weight value (Δw), find the resulting change in the output signal ($\Delta X3$), and divide the two ($\Delta X3/\Delta w$). Later in this chapter we will look at an example that calculates the slope this way. However, in the present example we will use a more efficient method.

Earlier we said that the nonlinearity (the sigmoid) needs to be *differentiable*. Here is where we will use this property. If we know the slope at each point on the nonlinearity, we can directly write an equation for the slope of each weight ($\Delta X3/\Delta w$) without actually having to perturb it. Consider a specific weight, for